MACHINE LEARNING

(CSDC-0334)

Lab File

B. Tech – CSE

6th Semester

Section-B

Department of Computer Science and Engineering
Dr. B. R. Ambedkar National Institute of Technology, Jalandhar,
Punjab, 144008

(January-June 2026)

Submitted to
Dr. Sourav Jindal

Submitted by:
Name : Tahira Mehta
Roll No : 23103146
Group : G7

ASSIGNMENT-4

Assignment: Apply linear regression for Loan amount prediction and visualize the interpretation.

Download the Loan Amount Prediction dataset from the link given below:

https://www.kaggle.com/datasets/phileinsophos/predict-loan-amount-data

Dataset contains records of the amount of loan that has been sanctioned to a user along with the features to predict the sanctioned amount.

Develop a python program to predict the loan amount to be sanctioned using Linear Regression (LR) Model using Scikit-learn library. Visualize the features from the dataset and interpret the results obtained by the model using Matplotlib library.

Use the following steps to do implementation:

1. Loading the dataset.
2. Pre-Processing the data (Handling missing values, Encoding, Normalization, Standardization).
3. Exploratory Data Analysis.
4. Feature Engineering techniques.
5. Split the data into training, testing and validation sets.
6. Train the model.
7. Test the model.
8. Measure the performance of the trained model.
9. Represent the results using graphs.

CODE:

```
# Installing Kaggle (if required)
!pip install kaggle

# Core Libraries
import numpy as np
import pandas as pd

# Visualization Libraries
import matplotlib.pyplot as plt
import seaborn as sb

# Sklearn Modules
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn import metrics

# Upload Dataset
```

```
from google.colab import files
files.upload()

# Load Dataset
data = pd.read_csv("train.csv")

data.head()
```

	Customer ID	Name	Gender	Age	Income (USD)	Income Stability	Profession	Type of Employment	Location	Loan Amount Request (USD)	...	Credit Score	No. of Defaults	Has Active Credit Card	Property ID	Property Age	Property Type	Property Location
0	C-36995	Frederica Shealy	F	56	1933.05	Low	Working	Sales staff	Semi-Urban	72809.58	...	809.44	0	NaN	746	1933.05	4	Rural
1	C-33999	America Calderone	M	32	4952.91	Low	Working	NaN	Semi-Urban	46837.47	...	780.40	0	Unpossessed	608	4952.91	2	Rural
2	C-3770	Rosetta Verne	F	65	988.19	High	Pensioner	NaN	Semi-Urban	45593.04	...	833.15	0	Unpossessed	546	988.19	2	Urban
3	C-26480	Zoe Chitty	F	65	NaN	High	Pensioner	NaN	Rural	80057.92	...	832.70	1	Unpossessed	890	NaN	2	Semi-Urban
4	C-23459	Afton Venema	F	31	2614.77	Low	Working	High skill tech staff	Semi-Urban	113858.89	...	745.55	1	Active	715	2614.77	4	Semi-Urban

```
print("Dataset Dimensions:", data.shape)

print("\nFeature Names:\n", list(data.columns))

print("\nData Types and Non-Null Count:\n")
data.info()

print("\nNull Values Count:\n")
print(data.isna().sum())
```

```
Null Values Count:

Customer ID                        0
Name                               0
Gender                            53
Age                                0
Income (USD)                    4576
Income Stability                1683
Profession                         0
Type of Employment              7270
Location                           0
Loan Amount Request (USD)          0
Current Loan Expenses (USD)      172
Expense Type 1                     0
Expense Type 2                     0
Dependents                      2493
Credit Score                    1703
No. of Defaults                    0
Has Active Credit Card          1566
Property ID                        0
Property Age                    4850
Property Type                      0
Property Location                356
Co-Applicant                       0
Property Price                     0
Loan Sanction Amount (USD)       340
dtype: int64
```

Handling Missing Values

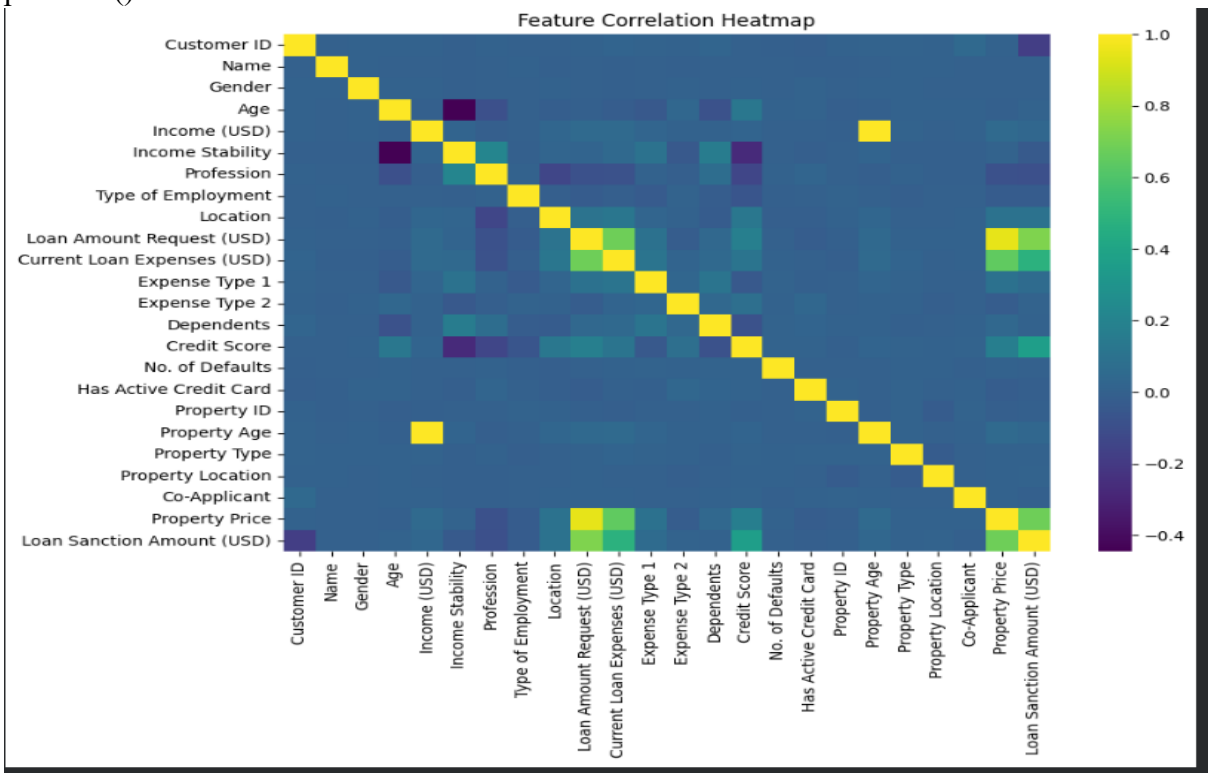
```
# Separate numeric and categorical columns
num_cols = data.select_dtypes(include=['int64','float64']).columns
cat_cols = data.select_dtypes(include=['object']).columns

# Fill numeric using median
data[num_cols] = data[num_cols].apply(lambda x: x.fillna(x.median()))

# Fill categorical using most frequent value
for c in cat_cols:
    data[c].fillna(data[c].value_counts().index[0], inplace=True)

print("Remaining Missing Values:\n", data.isnull().sum())
```

Encoding Categorical Features

```
encoder = LabelEncoder()

for column in cat_cols:
    data[column] = encoder.fit_transform(data[column])

data.head()
```

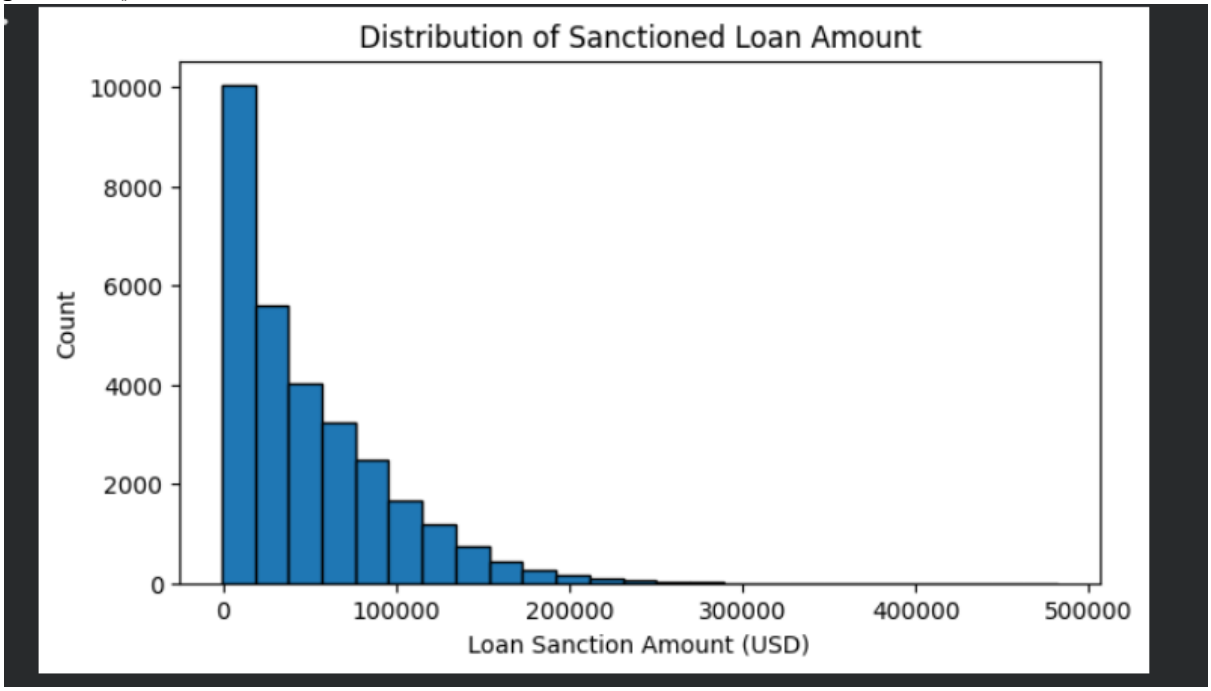
Correlation Analysis

```
plt.figure(figsize=(10,7))
sb.heatmap(data.corr(), cmap="viridis")
plt.title("Feature Correlation Heatmap")
plt.show()
```



Distribution of Target Variable

```
plt.figure(figsize=(7,4))
plt.hist(data["Loan Sanction Amount (USD)"], bins=25, edgecolor='black')
plt.title("Distribution of Sanctioned Loan Amount")
plt.xlabel("Loan Sanction Amount (USD)")
plt.ylabel("Count")
plt.show()
```



Feature Engineering

```
# Creating Financial Ratios

data["Request_to_Price"] = data["Loan Amount Request (USD)"] / data["Property Price"]

data["Income_Request_Ratio"] = data["Income (USD)"] / data["Loan Amount Request
(USD)"]

data.head()
```

Loan Sanction Amount (USD)	Request_to_Price	Income_Request_Ratio
54607.18	0.607083	0.026549
37469.98	0.854839	0.105747
36474.43	0.629385	0.021674
56040.54	0.659230	0.027760
74008.28	0.545908	0.022965

Define Independent and Dependent Variables

```
# Remove unnecessary identifiers
features = data.drop(columns=[
    "Loan Sanction Amount (USD)",
    "Customer ID",
    "Name",
    "Property ID"
])

target = data["Loan Sanction Amount (USD)"]

print("Independent Variables Shape:", features.shape)
print("Dependent Variable Shape:", target.shape)
```

```
•••  Independent Variables Shape: (30000, 22)
     Dependent Variable Shape: (30000,)
```

Splitting Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    features, target,
    test_size=0.25,
    random_state=1
)

print("Training Records:", X_train.shape[0])
print("Testing Records:", X_test.shape[0])
```

```
•••  Training Records: 22500
     Testing Records: 7500
```

Normalization

```
scale = StandardScaler()

X_train_scaled = scale.fit_transform(X_train)
X_test_scaled = scale.transform(X_test)
```

Model Building

```
lr_model = LinearRegression()

lr_model.fit(X_train_scaled, y_train)

print("Linear Regression Model Successfully Trained.")
```

Model Prediction

```
predicted_values = lr_model.predict(X_test_scaled)

print("Sample Predictions:\n", predicted_values[:8])
```

```
•••  Sample Predictions:
     [ 31731.29782273  71063.00505361  98999.62015111  44103.69196079
      21050.33386191  14333.94379606 125113.12203376  57993.19582866]
```

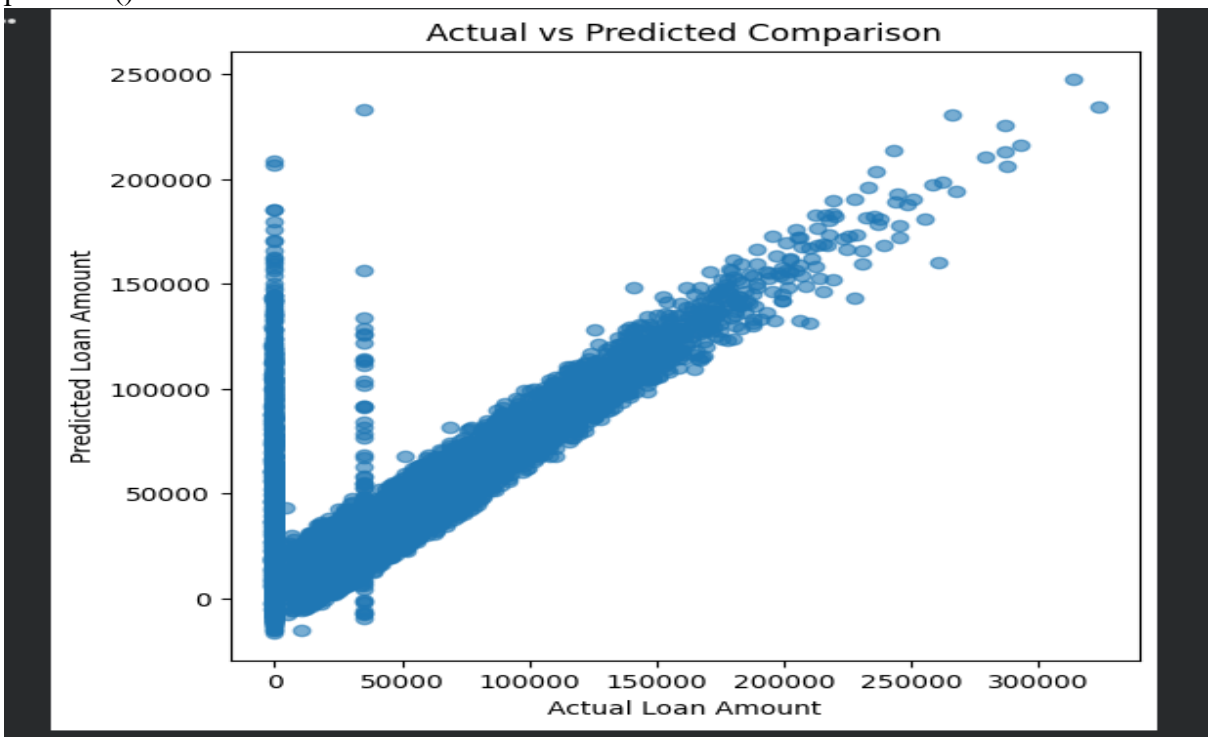
Performance Metrics

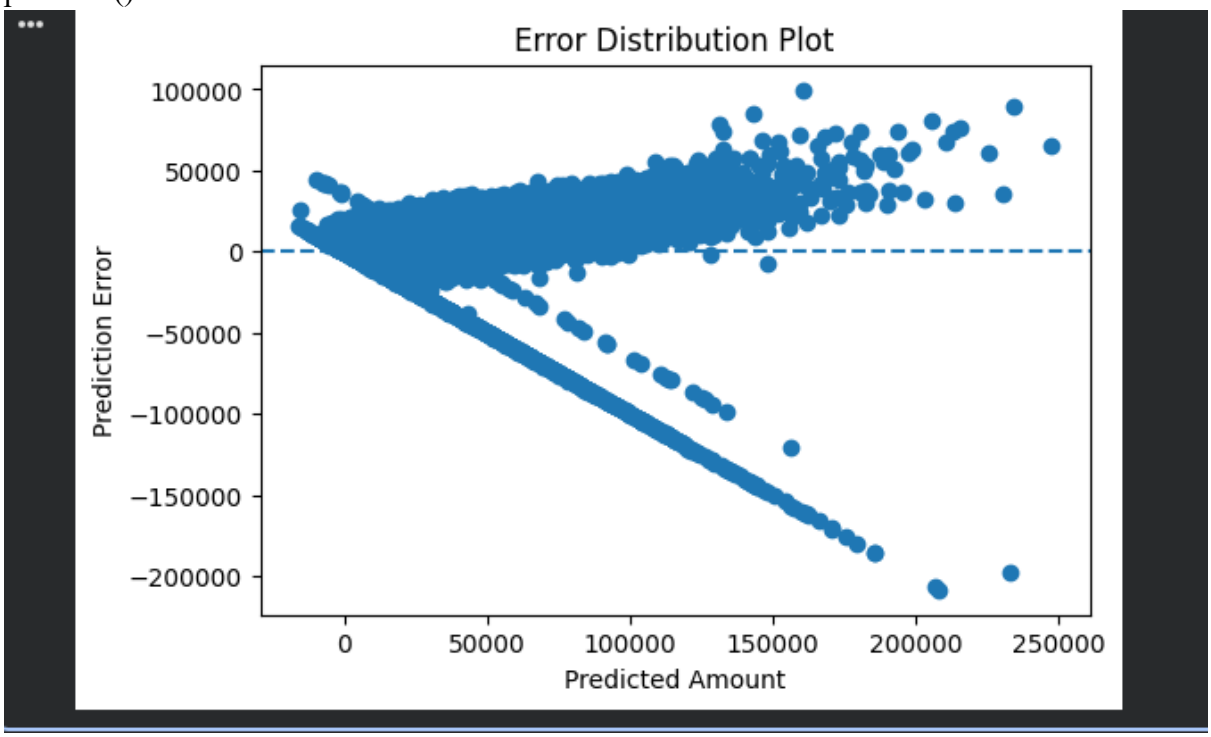
```
mse_value = metrics.mean_squared_error(y_test, predicted_values)
mae_value = metrics.mean_absolute_error(y_test, predicted_values)
r2_value = metrics.r2_score(y_test, predicted_values)

print("MSE:", mse_value)
print("MAE:", mae_value)
print("R² Score:", r2_value)
```

```
•••  MSE: 992005942.9462544
     MAE: 21755.032990273936
     R² Score: 0.5631653972154531
```

Actual vs Predicted Graph

```
plt.figure(figsize=(6,6))
plt.scatter(y_test, predicted_values, alpha=0.6)
plt.xlabel("Actual Loan Amount")
plt.ylabel("Predicted Loan Amount")
plt.title("Actual vs Predicted Comparison")
plt.show()
```



Error Distribution

```
error = y_test - predicted_values

plt.figure(figsize=(6,4))
plt.scatter(predicted_values, error)
plt.axhline(0, linestyle='--')
plt.xlabel("Predicted Amount")
plt.ylabel("Prediction Error")
plt.title("Error Distribution Plot")
plt.show()
```